

Azure-Based Data Integration Platform — Full Architecture Document

(Generated by ChatGPT — Enterprise Architecture Level)

1. Executive Summary

This document describes a complete, enterprise-grade, multi-tenant data integration platform designed to ingest, validate, transform, enrich, and deliver B2B/EDI/ecommerce/ERP data. It mirrors the capabilities of commercial platforms such as Dell Boomi, MuleSoft, ZEDI, and Celigo — but built entirely on modern Azure-native services.

The solution is: - **Fully serverless** - **Highly scalable** - **Zero-trust secure** - **Multi-tenant and partner-configurable** - **Low-code/no-code extensible** - **Event-driven** - **Optimized for EDI, XML, JSON, CSV, API, SFTP, AS2**

This architecture supports end-to-end integration flows from ingestion → orchestration → validation → transformation → business rules → storage → outbound delivery → monitoring.

2. High-Level Architecture Overview

The platform consists of six major layers: 1. **Ingestion Layer** (API, SFTP, AS2, Portal Upload) 2. **Orchestration Layer** (Event Grid, Service Bus, Durable Functions) 3. **Processing & Transformation Layer** (Container Apps Jobs, Mapping Engine, Rules Engine) 4. **Persistence Layer** (Blob Storage, Cosmos DB, Table Storage, Log Analytics, App Insights) 5. **Delivery Layer** (Logic Apps, Azure Functions, APIM, Notifications) 6. **Partner Feedback & Monitoring UI** (React + Static Web Apps) 7. **Cross-Cutting Concerns** (Security, CI/CD, Governance, Monitoring)

Every component communicates using correlation IDs and durable metadata stored in Cosmos DB.

3. Ingestion Layer

3.1 Supported Ingestion Methods

- **REST API** via Azure API Management
- **SFTP/FTPS** using Azure Blob SFTP Endpoint
- **AS2** via Logic Apps Integration Account
- **Manual upload** via Partner UI
- **Event-driven ingestion** from partner queues or webhooks

3.2 Responsibilities

- Authenticate inbound connections
- Perform lightweight preprocessing (normalizing filenames, generating correlation ID)

- Write raw files to `raw/` storage zone
- Emit Event Grid events for orchestration
- Log initial metadata in Cosmos DB

3.3 Technologies

- APIM
 - Logic Apps (AS2, SFTP)
 - Blob Storage (SFTP)
 - Event Grid
 - Cosmos DB
-

4. Orchestration Layer

4.1 Primary Components

- **Event Grid** (file arrival notifications)
- **Azure Functions** (event routers)
- **Service Bus** (work queues)
- **Durable Functions** (long-running workflows)

4.2 Key Responsibilities

- Determine *who* sent a file
- Determine *what flow* must execute
- Create processing messages for workers
- Update metadata status (e.g., "Received", "QueuedForProcessing")
- Handle routing for multi-partner logic

4.3 Sample Orchestration Message

```
{
  "filePath": "raw/partnerA/orders/file123.xml",
  "partnerId": "partnerA",
  "flowType": "order",
  "schemaVersion": "v2",
  "correlationId": "abc-123"
}
```

5. Processing & Transformation Layer

This is the heart of the system.

5.1 Compute Layer (Container Apps Jobs)

- Stateless .NET 8 workers
- Autoscale to 1000s of jobs

- Triggered from Service Bus
- Executes full processing pipeline

5.2 Format Detection

Techniques: - Magic byte scanning - First-line heuristics - Extension hints - EDI segment detection
(ISA*, UNB+)

Output informs validator + mapper selection.

5.3 Validation Layer

- **XML** → XSD validation
- **JSON** → JSON Schema
- **CSV** → header + column count + type rules
- **EDI** → Logic Apps B2B or .NET EDI libraries

Errors → errors/ folder and Cosmos metadata update.

5.4 Mapping Engine

Two options: 1. **Custom Fluent .NET Mapping DSL** 2. **Liquid Templates** (Logic Apps)

Mappings are versioned per partner in Cosmos DB.

5.5 Business Rules Engine

- Stores rules in Cosmos DB
- Uses C#, JMESPath, or JavaScript expressions
- Applies overrides, routing, transformations

5.6 Complete Sequence

1. Download raw file
2. Detect format
3. Load partner schema
4. Validate
5. Write to valid/
6. Load mapping version
7. Transform to canonical
8. Apply rules
9. Write output to transformed/
10. Publish "transformation complete" event
11. Delivery Layer executes

6. Persistence Layer

6.1 Blob Storage (ADLS Gen2)

Folder structure:

```
raw/  
valid/  
transformed/  
errors/  
archive/
```

6.2 Cosmos DB

Stores: - File metadata - Partner configuration - Mapping versions - Rules - Audit lifecycle

6.3 Table Storage

- Append-only audit logs
- Cheap long-term retention

6.4 Log Analytics

- Single source of truth for logs
- KQL dashboards
- Failure heatmaps

6.5 Application Insights

- Distributed tracing
- Live metrics
- Correlation across components

7. Delivery Layer

7.1 Logic Apps (Standard)

Used for: - Outbound API calls - SaaS connectors - EDI (X12/EDIFACT) outbound - OAuth2 authentication
- Advanced workflows

7.2 API Management (APIM)

Used for: - Request signing - Partner-specific policies - Throttling outbound delivery - Mock/test endpoints

7.3 Azure Functions (Custom Delivery Code)

Used for: - SFTP uploads - API webhooks - File generation (CSV/XML/EDI) - Retry/backoff logic

7.4 Notification Mechanisms

- SendGrid email
- Teams/Slack webhooks
- Azure Notification Hub (mobile)

7.5 Delivery Patterns

- Direct push API
 - SFTP drop
 - Event-based publish
 - AS2/EDI delivery
 - Batch delivery windows
-

8. Partner Feedback & Monitoring UI

8.1 Hosted on

- Azure Static Web Apps (recommended)
- Optionally App Service (for SSR/Next.js)

8.2 UI Capabilities

- File explorer (raw/valid/transformed)
- End-to-end pipeline visualization
- Replay/resubmit flows
- Search (by partner, PO, correlation ID, time range)
- Download files
- Mapping version display
- Rules that fired
- API rate limits & throughput
- Delivery status tracking

8.3 Authentication & Authorization

- Entra ID
- RBAC roles:
 - PartnerUser
 - PartnerAdmin
 - InternalOps
 - SystemAdmin
- Claims enforcement in API layer

8.4 Live Metrics

- App Insights charts
 - Log Analytics dashboards
 - Queue depth
 - Failure rates
 - SLA measurement
-

9. Security & Governance Layer

9.1 Identity & Access Management

- Entra ID for all identities
- Managed Identity for services
- APIM OAuth2 enforcement
- Key Vault for secrets

9.2 Network Security

- Private Endpoints
- VNET-injected Container Apps
- Firewall-restricted Cosmos DB
- IP allowlists for SFTP/AS2 partners

9.3 Data Security

- At-rest encryption (AES-256)
- In-transit TLS 1.2+
- Sensitive field masking in logs
- PII redaction in UI

9.4 Compliance (optional)

- Immutable blob storage
 - Audit retention policies
 - Evidence export for SOC2/ISO27001
-

10. Error Handling & Retry Architecture

10.1 Error Types

- Validation errors
- Mapping errors
- Business rule failures
- Delivery failures
- Partner API downtime
- Poison messages in queues

10.2 Handling Mechanisms

- Service Bus DLQs
- Auto-retry with exponential backoff
- Circuit breaker for API outages
- DLQ replay tool in Partner UI
- Cosmos DB stores full error trace
- Errors written to `errors/` folder

10.3 Notification Strategy

- Email to partner
 - Slack/Teams alert to Ops
 - Incident automatically created (optional)
-

11. CI/CD & Environment Strategy

11.1 Infrastructure as Code

- Bicep or Terraform
- GitOps pipelines

11.2 Deployment Flow

- PR → Build → Test → Deploy to Dev → UAT → Prod
- GitHub Actions or Azure DevOps

11.3 Environment Separation

- Dev
- Test
- UAT
- Production
- Optional Partner-Specific Sandboxes

11.4 Versioning Strategy

- Each mapping stored with version tags
 - Semantic versioning (v2.3, v4.1)
 - Promote mapping versions via CI/CD
-

12. Partner Onboarding Workflow

1. Create partner record in Cosmos DB
2. Upload schemas (XSD/JSON)
3. Define validation rules
4. Configure mapping version
5. Configure delivery endpoint
6. Assign UI access roles
7. Run test file
8. Promote to production

Partner onboarding can be fully UI-driven for self-service.

13. End-to-End Flow Summary

1. File Arrives

API, SFTP, AS2, or UI.

2. Orchestration Triggered

Event Grid → Service Bus → Durable Function.

3. Processing Job Executes Validation + Mapping + Rules.

4. Output Stored & Metadata Updated

Blob + Cosmos DB.

5. Delivery Layer Sends Payload

API, SFTP, EDI, webhook.

6. Monitoring UI Displays Status

Pipeline, errors, metrics.

7. Alerts Triggered

Email, Slack, Teams.

14. Conclusion

This architecture represents a modern, battle-tested, scalable, secure, multi-tenant Azure-based integration platform capable of powering enterprise-grade B2B, EDI, API, ecommerce, and ERP integrations.

Built with the same philosophy as leading commercial platforms — but using native cloud primitives — it provides: - Maximum scalability - Minimal operational burden - Full traceability - Deterministic behavior - Multi-tenant extensibility - Business-friendly monitoring & replay tools

It is a platform designed not just for **integration**, but for **growth**, **automation**, and **partner enablement**.

Appendix A — Detailed Architecture Blueprint Expansion

(Approx. 30–40 page equivalent when exported to PDF)

A1. Business Requirements & Goals

A1.1 Core Business Drivers

- Reduce partner onboarding time from weeks to hours.

- Support semi-technical users (Ops/Support) with low-code tools.
- Standardize all data flows into canonical models.
- Guarantee end-to-end traceability for compliance & audits.
- Provide predictable SLAs for mission-critical integrations.
- Offer multi-tenant, isolated partner configuration.
- Minimize operational burden using serverless components.

A1.2 Architectural Goals

- Cloud-native, event-driven, fully scalable.
- Strict separation of duties for ingestion, processing, and delivery.
- Strong observability and replay capability.
- Support 100+ partners with up to 10k files/hr each.
- Deterministic transformations (versioned, reproducible).

A2. Domain Model & Canonical Modeling Strategy

A2.1 Purpose of Canonical Models

Canonical models allow the platform to: - Normalize disparate partner formats. - Reduce transformation complexity. - Create reusable mapping pipelines. - Decouple inbound and outbound schemas.

A2.2 Canonical Model Taxonomy

1. **Order Model** (PO, SO, marketplace orders)
2. **Inventory Model** (stock levels, availability)
3. **Shipment Model** (ASN, tracking, fulfillment)
4. **Invoice Model** (AR/AP documents)
5. **Product Model** (catalog, pricing, attributes)
6. **Status/Feedback Model** (acknowledgments, MDNs)

A2.3 Canonical Versioning

Each canonical model is versioned:

v1 → v2 → v3

Partners may use a different canonical version per flow.

A3. Detailed Component Interactions

A3.1 Ingestion → Orchestration Interaction

1. File upload is authenticated via APIM.
2. File lands in Blob SFTP or API upload endpoint.
3. Event Grid emits a “BlobCreated” event.
4. Orchestration Function determines partner + flow.

5. Creates a command message in Service Bus.

A3.2 Orchestration → Processing Interaction

- Durable Functions supervise long-running workflows.
- Ensures retries and compensating actions.
- Writes intermediate states to Cosmos DB.

A3.3 Processing → Persistence Interaction

- Workers write validated → `valid/`.
 - Transform outputs → `transformed/`.
 - Errors → `errors/`.
 - Metadata updated after every step.
-

A4. Transformation Engine Deep Dive

A4.1 Mapping DSL Design

A custom .NET fluent DSL supports: - Conditional expressions - LINQ-based transformation pipelines - Reusable sub-maps - Mapping inheritance (e.g., common header logic)

A4.2 Liquid Template Integration

Used when: - Non-technical users modify mappings - Logic Apps orchestrate full transformations

A4.3 Performance Optimization

- Streaming transformations (XML/JSON readers)
- Avoid in-memory DOM loading for large files
- Partition-heavy operations for CSV

A4.4 Parallel Execution

- Multi-threaded line/item processing
 - Fan-out for large orders with thousands of items
-

A5. Business Rules Engine Deep Dive

A5.1 Rule Types

1. **Validation rules** (required fields, value constraints)
2. **Transformation rules** (modify fields, add defaults)
3. **Routing rules** (change outbound system based on data)
4. **Enrichment rules** (lookup tables, partner references)

A5.2 Rule Execution Model

- Rules executed post-mapping on the canonical payload.

- Rules evaluate using a pluggable expression engine.
- Results appended to metadata.

A5.3 Rule Management Lifecycle

- Versioned in Cosmos DB
 - Can be updated without code deployment
 - Sandbox mode for testing rules with sample payloads
-

A6. Data Lake Strategy

A6.1 Storage Zones

- **Raw** → Exact partner input
- **Validated** → Schema-verified files
- **Transformed** → Canonical or outbound format
- **Errors** → Failed transformations with logs
- **Archive** → Long-term storage with immutability

A6.2 Tiering Strategy

- Hot tier for recent files (7–30 days)
 - Cool tier for medium retention (90–180 days)
 - Archive tier beyond 1 year
-

A7. Observability Architecture

A7.1 Logging Types

- Application logs (Functions, Logic Apps, Containers)
- Metadata logs (Cosmos DB)
- Security logs (APIM, Entra ID)
- Audit logs (Table Storage)

A7.2 Correlation ID Strategy

IDs propagate across all services:

API → Orchestration → Worker → Delivery → UI

A7.3 Dashboards

- File throughput
- Transform latency
- Delivery success rates
- API error heatmaps
- Queue depth graphs

A8. Security Architecture

A8.1 Identity

- System identities via Managed Identity
- User identities via Entra ID

A8.2 Network Boundaries

- All storage accounts private
- Service Bus + Cosmos DB behind Private Endpoints

A8.3 Encryption

- Key Vault-managed keys
- Double encryption optional for regulated industries

A8.4 Threat Protection

- APIM rate limiting
- BOT & anomaly detection
- AS2 signature & encryption requirements

A9. CI/CD & DevOps Blueprint

A9.1 Repo Structure

```
/infra
/functions
/container-apps
/mappings
/rules
/ui
/tests
```

A9.2 Pipelines

- Validate IaC
- Build containers
- Deploy Logic Apps Standard with parameters
- Sync mappings to Cosmos DB

A9.3 Quality Gates

- Unit tests for mappings
 - Integration tests for sample payloads
 - Contract tests for partner API schemas
-

A10. SLA & Reliability Engineering

A10.1 SLAs

- Ingestion: <2 seconds
- Validation: <30 seconds
- Transformation: <20 seconds for 95th percentile
- Delivery: partner-dependent

A10.2 Resilience Patterns

- Retry w/ exponential backoff
- Dead-letter queues
- Circuit breakers for outbound APIs
- Automatic rehydration of Durable Functions

A10.3 Disaster Recovery

- Geo-redundant storage (GRS)
 - Zone-redundant Service Bus
 - Multi-region failover for Cosmos DB
-

A11. Partner Onboarding Playbook

A11.1 Onboarding Phases

1. Discovery
2. Configuration
3. Validation
4. Certification
5. Production Go-Live

A11.2 UI-driven Setup

Partnership defined via: - Mapping version selection - Rule set selection - Delivery endpoint configuration - API key & credentials

A11.3 Automated Testing

- Upload sample files
 - Validate transformations
 - Compare expected vs. actual outputs
-

A12. Example End-to-End Use Case

"Partner A sends EDI 850 Purchase Order"

1. EDI 850 received via AS2
2. Stored in `raw/partnerA/orders/`

3. Event Grid triggers orchestration
 4. EDI validated via Integration Account
 5. Mapping applied to canonical Order model
 6. Business rules apply tax & routing logic
 7. Output JSON written to transformed/
 8. Delivery Logic App sends to SAP
 9. SAP OK → acknowledgment generated
 10. Partner UI shows "Delivered Successfully"
-

A13. Future Enhancements

- AI-assisted mapping generation
 - ML-based anomaly detection for files
 - Auto-scaling prediction for peak loads
 - Partner self-service mapping editor
 - Unified integration marketplace
-

END OF BLUEPRINT